PyRINEX 4.0 Manual

A Python toolkit for batch processing and quality checking of RINEX 2/3 GNSS data

Han Jinzhen
hanjinzhen9@gmail.com

May 8, 2026

Contents

1. Introduction

PyRINEX is a Python package for batch processing and quality-checking GNSS observation files in the RINEX 2.xx and 3.xx formats. It targets two use cases that occur repeatedly in geodetic and surveying workflows: (i) consolidating large, inconsistently-named archives of RINEX files into a clean directory tree, and (ii) running standard quality-check products—multipath, ionospheric

drift, cycle-slip detection, sky plot, signal plot—on every file in a folder without having to drop into a GUI tool.

The package is released under the Apache License 2.0; the canonical citation is the PeerJ Computer Science paper that introduced it [2].

This manual documents PyRINEX 4.0, which is a substantial rewrite of the v3.x release. The high-level API and command surface are backwards-compatible (legacy names continue to work and emit `DeprecationWarning`), but every public function now returns plain Python objects rather than JSON strings, the package layout has been split into focused submodules, the typo'd module name `QulityCheck` has been corrected to `quality_check`, and a number of bugs identified post-publication have been fixed (see §8).

2. Installation

PyRINEX is published on PyPI:

```
pip install PyRINEX
```

To install the development version from source:

```
git clone https://github.com/geumjin99/PyRINEX
cd PyRINEX
pip install -e .[dev]
```

Python 3.9 or newer is required. Runtime dependencies are `numpy`, `matplotlib`, `seaborn`, `chardet`, and `python-dateutil`. The development extras add `pytest` and `ruff` for tests and linting.

3. Quick start

After installation, import the modules and call them on a RINEX file:

```
from PyRINEX import reader, data_management, quality_check

header = reader.read_obs_header("test0010.24o")
obs = reader.read_obs("test0010.24o")
nav = reader.read_nav("test0010.24n")

quality_check.quality_check("test0010.24o")
```

The next sections describe each module in detail.

4. Reader module

The `PyRINEX.reader` module turns RINEX text files into native Python data structures. Three high-level functions cover the typical needs:

Function	Description
`read_obs_header(path)`	Parse the header section of an observation file; returns an `ObsHeader` dataclass.
`read_obs(path)`	Parse the body of an observation file; returns a dict mapping epoch-label strings to per-PRN observation dicts.
`read_nav(path)`	Parse a GPS broadcast-navigation file; returns a dict mapping each PRN to a list of broadcast ephemeris records.

`read_nav_iono(path)`	Extract the eight Klobuchar ionosphere model parameters from a GPS navigation file.

4.1. Header

`ObsHeader` (defined in `PyRINEX.reader`) is a frozen dataclass with the following fields:

Field	Meaning
`version`	RINEX major version (`2` or `3`).
`type`	Observation system letter (`G`, `M`, ...).
`MARKER_NAME`	`[line, value]`; line index is preserved so `clean_rinex` can rewrite this header line in place.
`MARKER_NUMBER`	As above.
`receiver_type` / `antenna_type`	As above.
`APPROX_POSITION_XYZ`	ECEF coordinates as a 3-tuple of floats, in metres.
`TIME_OF_FIRST_OBS` / `TIME_OF_LAST_OBS`	Hyphen-joined time strings; both are kept distinct (a bug in v3.x clobbered one with the other).
`ObsTypes`	List of observation codes for v2; per-system dict for v3.
`PRNS`	Sorted list of every PRN that appears in the body of the file.
`END_OF_HEADER`	Line index of `END OF HEADER`.

4.2. Observations

`read_obs` returns a `dict` keyed by epoch label ("`yy mm dd hh mi ss.ss`"). Each value is itself a dict containing the entry `sat_num` and one entry per PRN whose value is a dict from observation-type to the verbatim 16-character RINEX field string. Values that are missing in the file are substituted with the sentinel `"  0.000  "`. Downstream quality-check routines treat this sentinel as "no observation".

4.3. Navigations

`read_nav` returns a dict from PRN to a list of broadcast ephemerides. Each ephemeris is a dict whose keys are descriptive parameter names ("Toe Time of Ephemeris", "sqrt(A)", "Cuc", ...) and whose values are the verbatim 19-character fields straight from the navigation file. To convert a field to a floating-point value, use the `_to_float` helper from `PyRINEX.orbit` (which understands Fortran-style "D" exponents):

```
from PyRINEX.orbit import _to_float
toe = _to_float(nav["G01"][0]["Toe Time of Ephemeris"])
```

4.4. Legacy JSON API

For backwards compatibility with code written against v3.x, the following functions return JSON-encoded strings rather than dicts: `oheader(path)`, `observations(path)`, `navigations(path)`, `nheader(path)`. They emit `DeprecationWarning` on every call. New code should prefer the "`read_*`" functions which return native objects directly and avoid the JSON round-trip entirely.

5. DataManagement module

`PyRINEX.data_management` provides bulk file-discovery and RINEX-cleaning helpers.

5.1. find_rinex (formerly DataFinding)

Given a root directory, a list of folder-name keywords and a file extension, `find_rinex` returns every matching path:

```
from PyRINEX.data_management import find_rinex

paths = find_rinex(
    root_path="/data/2008",
    keywords=["Gyeong-gi", "Unified control points"],
    extension=".08o",
)
```

The match is case-insensitive on the extension; the keywords must all be substrings of the folder path of each candidate file.

5.2. clean_rinex (formerly DataCleaning)

`clean_rinex` performs five corrections on a list of RINEX observation files and copies the matching navigation file alongside:

1. Truncate the marker name to four characters (right-padding with "a" if necessary).
2. Substitute receiver-type strings according to a user-provided `ReceiverLibrary.csv`.
3. Substitute antenna-type strings according to `AntennaLibrary.csv`.
4. Optionally fill in `NONE` as the antenna radome.
5. Replace non-ASCII characters in the first twenty header lines with "-".

The library CSVs are simple two-column tables of the form "`wrong?right`"—one entry per line, comma-free in either column—which lets users add their own entries without touching PyRINEX's source. A typical receiver library entry:

```
TRIMBLENETR9 ?TRIMBLE NETR9
TPS GB1000 ?TPS GB-1000
```

After running, `clean_rinex` writes a CSV report at *output_root*`/report.csv` summarising every file processed, its origin and target paths, and the changes that were made.

5.3. Coordinate conversion

The WGS-84 ECEF↔BLH conversions used by `clean_rinex` are exposed independently in `PyRINEX.coordinates`:

```
from PyRINEX.coordinates import xyz_to_blh, blh_to_xyz

lon, lat, h = xyz_to_blh(-3173543.0, 4134173.3, 3666275.5,
                         radians=False)
```

The forward conversion uses the closed-form Bowring iteration with a 0.1 m convergence threshold on height.

6. QualityCheck module

`PyRINEX.quality_check` provides every per-file quality-check routine and the two batch drivers `quality_check` and `batch_quality_check`.

Function	Description
`satellite_signal_plot(path)`	Per-PRN signal-presence raster (Figure **??**).
`azimuth_elevation(path)`	Azimuth and elevation of every GPS satellite per epoch. Writes a sky-plot PNG and a CSV.
`multipath_iod(path)`	MP1 / MP2 multipath, ionospheric delay (ION) and drift (IOD). Writes four PNGs and two CSVs.
`cycle_slip(path)`	Carrier-phase second-difference cycle-slip indicator. Writes one PNG and one CSV.
`quality_check(path)`	Run all four routines on a single file.
`batch_quality_check(root, keywords, ext)`	Run `quality_check` on every file matching the `find_rinex` criteria of §5.1.

6.1. Multipath observable

For each constellation supported by `frequency_pair` (GPS, GLONASS, Galileo, SBAS), the L1 and L2 multipath observables are computed from the dual-frequency code–phase combination [1]:

$$\mathrm{MP}_1 = P_1 - \left(\frac{2}{\gamma - 1} + 1\right)\lambda_1 L_1 + \frac{2}{\gamma - 1}\lambda_2 L_2, \tag{1}$$

$$\mathrm{MP}_2 = P_2 - \frac{2\gamma}{\gamma - 1}\lambda_1 L_1 + \left(\frac{2\gamma}{\gamma - 1} - 1\right)\lambda_2 L_2, \tag{2}$$

where $\gamma = (f_1/f_2)^2$. Per continuous arc the per-arc mean of MP1 and MP2 is removed, isolating the multipath component from the arbitrary integer ambiguity bias.

6.2. Geometry-free combination and ionospheric drift

The geometry-free phase combination

$$\mathrm{ION} = \frac{\gamma}{\gamma - 1}(\lambda_1 L_1 - \lambda_2 L_2) \tag{3}$$

is differenced between epochs to produce the ionospheric drift indicator $\mathrm{IOD} = d\mathrm{ION}/dt$.

6.3. Cycle-slip detector

PyRINEX uses the second-difference of the geometry-free combination to flag cycle slips in carrier phase:

$$\Delta^2\Phi_{\mathrm{IF}}(t) = \Phi_{\mathrm{IF}}(t-1) - 2\Phi_{\mathrm{IF}}(t) + \Phi_{\mathrm{IF}}(t+1). \tag{4}$$

A non-zero second difference outside numerical noise is taken to mean either a slip or a code-range outlier; the latter is rejected by an absolute-value threshold.

6.4. GPS satellite position

The azimuth/elevation product depends on the ECEF position of each GPS satellite, computed from the broadcast ephemeris using the algorithm in IS-GPS-200, Table 20-IV. PyRINEX exposes that calculation as `PyRINEX.orbit.gps_satellite_position(eph, t_k)` so users can call it directly:

```python
from PyRINEX.reader import read_nav
from PyRINEX.orbit import gps_satellite_position

nav = read_nav("test0010.24n")
x, y, z = gps_satellite_position(nav["G01"][0], t_k=0.0)
```

6.5. Output products

When run on a file `stub.24o`, `quality_check` writes the following sibling files (matching the published manual screenshot):

- `stubSignalPlot.jpg`
- `stubskyplot.png` (only when a matching nav file is found)
- `stubaziele.csv`
- `stubMP1_plot.png` / `stubMP2_plot.png`
- `stubION_plot.png` / `stubIOD_plot.png`
- `stubmp1mp2.csv` / `stubIonIod.csv`
- `stubCycleSlipCarrier.png` / `stubCScarrier.csv`

7. Practical use cases

7.1. Bulk data cleaning

Suppose you have a folder hierarchy `2008/<region>/<point_type>/<date>/RINEX/<station>/` that contains observation files with marker names that are too long, non-ASCII characters in the header, and inconsistent receiver-type spellings. The following script consolidates the Gyeong-gi "Unified control points" subset into a tidy date-indexed tree:

```python
from PyRINEX.data_management import clean_rinex, find_rinex

paths = find_rinex(
    root_path="/data/2008",
    keywords=["Gyeong-gi", "Unified control points"],
    extension=".08o",
)
clean_rinex(
    rinex_files=paths,
    receiver_library_path="ReceiverLibrary.csv",
    antenna_library_path="AntennaLibrary.csv",
    output_root="/data/2008-clean",
    radome=True,
)
```

7.2. Batch quality check

Once a folder is clean, run quality-check on every file:

```python
from PyRINEX.quality_check import batch_quality_check

batch_quality_check(
```

```
    root_path="/data/2008-clean",
    keywords=[],
    extension=".08o",
)
```

8. Changelog

4.0.0

- Module `QulityCheck` has been renamed to `quality_check` (typo fix). The old name is preserved as a deprecation shim.
- Module `DataManagement` has been renamed to `data_management` (PEP-8 naming). The old name is preserved as a deprecation shim.
- `read_obs_header` / `read_obs` / `read_nav` return native Python objects instead of JSON strings. The legacy `oheader` / `observations` / `navigations` helpers continue to return JSON for callers that wrap them with `json.loads`.
- Bug: `TIME OF LAST OBS` no longer overwrites `TIME OF FIRST OBS`.
- Bug: `abs(MP1 > threshold)` was an always-true `abs(bool)`; corrected to `abs(MP1) > threshold`.
- Bug: hard-coded "\\" path separators in the cleaning routine have been replaced with `os.path.join` (now works on Linux and macOS).
- Bug: the inner-`return` indentation in `file_name_extesion_judgement` that caused only the first directory to be scanned has been fixed.
- Bug: `new_marker.replace(" ", "a")` was a no-op (strings are immutable); the result is now assigned back.
- Bug: aliasing between `prns` and `fieldnames` in the multipath/cycle-slip output writers has been removed.
- The four hand-unrolled per-constellation extraction blocks in `LandC` have been replaced by a config-driven loop.
- Distribution metadata has moved from `setup.py` to `pyproject.toml` (PEP 621). The `yourusername` placeholder URL has been corrected.
- Tests: a `pytest` suite with synthetic RINEX 2/3 fixtures runs end-to-end without external data.

References

[1] Lou H. Estey and Charles M. Meertens. TEQC: The multi-purpose toolkit for GPS/GLONASS data. In *GPS Solutions*, volume 3, pages 42–49. Springer, 1999.

[2] Jinzhen Han, Seung Jun Lee, Hong Sik Yun, Kwang Bae Kim, and Sang Won Bae. PyRINEX: a new multi-purpose Python package for GNSS RINEX data. *PeerJ Computer Science*, 10:e1800, 2024.